МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: К. Смирнов
Группа: М8О-ХХХБ-ХХ
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков в управлении процессами в ОС и в обеспечении обмена данных между процессами посредством каналов.

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Группа вариантов 5. Родительский процесс создаёт два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процессы child1 и child2 производят работу над строками и пишут результаты своей работы в стандартный вывод.

Вариант: 20. Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки.

Метод решения

Программа состоит из двух исполняемых файлов: `parent` и `child`.

Родительский процесс сначала читает со стандартного ввода два имени файла и открывает их на запись вызовом `open` с флагами `O_WRONLY | O_CREAT | O_TRUNC` и правами `0644`. Затем он создаёт два канала `pipe1` и `pipe2` и два дочерних процесса через `fork`.

В каждом дочернем процессе вызовом `dup2` стандартный ввод перенаправляется на читающий конец своего канала, а стандартный вывод на свой файл. После этого все лишние дескрипторы закрываются, и вызовом `execl` образ процесса заменяется на программу `child`. Так `child` ничего не знает о каналах и файлах: он просто читает строки из `stdin` и пишет перевёрнутые строки в `stdout`.

Родитель закрывает у себя читающие концы каналов и дескрипторы файлов, потом читает строки функцией `getline`. Если длина строки без символа перевода строки больше 10, строка пишется в `pipe2`, иначе в `pipe1`. Длина считается в байтах.

Когда ввод закончился (EOF), родитель закрывает пишущие концы каналов. Дочерние процессы получают EOF при чтении, завершаются, а родитель дожидается их через `waitpid` и выводит код завершения. Поэтому зомби-процессов не остаётся.

Важный момент: каждый процесс должен закрыть все ненужные ему концы каналов. Если хотя бы один пишущий конец канала останется открытым (например, в другом дочернем процессе, который получил его по наследству после `fork`), читающий процесс никогда не получит EOF и будет ждать бесконечно. Поэтому в `spawn_child` передаётся массив дескрипторов, которые нужно закрыть в ребёнке.

Сигнал `SIGPIPE` в родителе игнорируется: если дочерний процесс неожиданно завершится, запись в канал вернёт ошибку `EPIPE`, и родитель обработает её, а не будет убит сигналом.

Описание программы

Проект собирается через CMake и разделён на файлы:

- `include/utils.h`, `src/utils.c` — общие функции для обеих программ:
  - `die(msg)` — выводит ошибку через `perror` и завершает процесс;
  - `write_all(fd, buf, len)` — пишет буфер целиком, повторяя `write`, если записалось не всё или вызов прерван сигналом (`EINTR`);
  - `reverse_string(str, len)` — переворачивает строку на месте;
  - константа `LENGTH_THRESHOLD = 10` — порог длины строки.
- `src/parent.c` — родительский процесс:
  - `spawn_child` — создаёт дочерний процесс: `fork`, `dup2` для `stdin` и `stdout`, закрытие лишних дескрипторов и `execl`;
  - `read_filename` — читает имя файла со стандартного ввода;
  - `open_output` — открывает файл на запись;
  - `wait_child` — ждёт процесс через `waitpid` (с повтором при `EINTR`) и печатает, как он завершился;
  - `main` — создаёт каналы и процессы, распределяет строки по каналам и дожидается детей.
- `src/child.c` — дочерний процесс: читает строки из `stdin` через `getline`, переворачивает их (без символа `\n`) и пишет в `stdout`.

Путь к `child` передаётся в `parent` при сборке через макрос `CHILD_PATH`, поэтому `parent` можно запускать из любой папки.

Используемые системные вызовы:

- `fork()` — создание дочернего процесса;
- `execl()` (`execve`) — замена образа процесса на программу `child`;
- `pipe()` — создание неименованного канала;
- `dup2()` — перенаправление дескриптора на `stdin`/`stdout`;
- `open()`, `close()` — открытие файлов и закрытие дескрипторов;
- `read()` (через `getline`), `write()` — чтение и запись;
- `waitpid()` — ожидание завершения дочернего процесса;
- `getpid()` — получение PID для вывода;
- `signal()` — игнорирование `SIGPIPE`;
- `exit()`, `_exit()` — завершение процесса.

Результаты

Сборка и запуск:

```
cmake -S . -B build
cmake --build build
./build/parent < input.txt
```

Входные данные (первые две строки — имена файлов для child1 и child2):

```
out1.txt
out2.txt
hello
hello world 12345
os lab 2
this line is longer than ten
abc
```

Вывод программы. Видно PID родителя и обоих дочерних процессов и то, что оба ребёнка завершились с кодом 0:

```
parent: PID=3025
File for child1 (lines <= 10 chars): File for child2 (lines > 10 chars): child: PID
    =3026
child: PID=3027
parent: started child1, PID=3026
parent: started child2, PID=3027
parent: child1 (PID=3026) exited with code 0
parent: child2 (PID=3027) exited with code 0
```

Содержимое файлов после работы. Строки длиной не больше 10 символов (`hello`, `os lab 2`, `abc`) попали к child1, длинные — к child2, все строки перевёрнуты:

```
$ cat out1.txt
olleh
2 bal so
cba
$ cat out2.txt
54321 dlrow olleh
net naht regnol si enil siht
```

После завершения программы зомби-процессов не остаётся: родитель дожидается обоих детей через `waitpid`, что видно и в выводе strace (вызовы `wait4`).

Выводы

В ходе работы я научился создавать процессы через `fork()` и `execl()`, перенаправлять их ввод-вывод через `dup2()` и передавать данные между процессами по каналам `pipe()`. Также я понял, что неиспользуемые концы каналов нужно закрывать, иначе читающий процесс не получит EOF и зависнет.

Исходная программа

```c
#ifndef UTILS_H
#define UTILS_H

#include <stddef.h>
#include <sys/types.h>

#define LENGTH_THRESHOLD 10

void die(const char *msg);

int write_all(int fd, const void *buf, size_t len);

void reverse_string(char *str, size_t len);

#endif
```

Листинг 1: include/utils.h

```c
#include "utils.h"

#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int write_all(int fd, const void *buf, size_t len) {
    const char *p = buf;
    size_t i = 0;
    while (i < len) {
        ssize_t batch = write(fd, p + i, len - i);
        if (batch == -1){
            if(errno == EINTR){
                continue;
            }
            return -1;
        }
        i += (size_t)batch;
    }
    return 0;
}

void reverse_string(char *str, size_t len) {
    if(len == 0){
        return;
    }
    size_t i = 0;
    size_t j = len - 1;
    while (i < j){
        char tmp = str[i];
        str[i] = str[j];
```

```
        str[j] = tmp;
        i++;
        j--;
    }
}
```

Листинг 2: src/utils.c

```
#include "utils.h"

#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#ifndef CHILD_PATH
#define CHILD_PATH "./child"
#endif

static pid_t spawn_child(int in_fd, int out_fd, const int *close_fds, size_t n_close)
    {
    pid_t pid = fork();
    if (pid == -1) {
        die("fork");
    }
    if (pid > 0) {
        return pid;
    }

    if (dup2(in_fd, STDIN_FILENO) == -1) {
        perror("dup2 stdin");
        _exit(EXIT_FAILURE);
    }
    if (dup2(out_fd, STDOUT_FILENO) == -1) {
        perror("dup2 stdout");
        _exit(EXIT_FAILURE);
    }

    close(in_fd);
    close(out_fd);
    for (size_t i = 0; i < n_close; i++) {
        close(close_fds[i]);
    }

    execl(CHILD_PATH, "child", (char *)NULL);

    perror("execl");
    _exit(EXIT_FAILURE);
}

static char *read_filename(const char *prompt) {
    printf("%s", prompt);
    fflush(stdout);
```

```
49
50      char *name = NULL;
51      size_t cap = 0;
52      ssize_t len = getline(&name, &cap, stdin);
53      if (len == -1) {
54          free(name);
55          if (ferror(stdin)) {
56              die("getline");
57          }
58          fprintf(stderr, "parent: expected a file name, got end of input\n");
59          exit(EXIT_FAILURE);
60      }
61      if (name[len - 1] == '\n') {
62          name[len - 1] = '\0';
63      }
64      return name;
65  }
66
67  static int open_output(const char *name) {
68      int fd = open(name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
69      if (fd == -1) {
70          die(name);
71      }
72      return fd;
73  }
74
75  static int wait_child(pid_t pid, const char *name) {
76      int status;
77      while (waitpid(pid, &status, 0) == -1) {
78          if (errno != EINTR) {
79              perror("waitpid");
80              return -1;
81          }
82      }
83      if (WIFEXITED(status)) {
84          printf("parent: %s (PID=%d) exited with code %d\n", name, pid, WEXITSTATUS(
        status));
85          return WEXITSTATUS(status) == 0 ? 0 : -1;
86      }
87      if (WIFSIGNALED(status)) {
88          printf("parent: %s (PID=%d) killed by signal %d\n", name, pid, WTERMSIG(status
        ));
89      }
90      return -1;
91  }
92
93  int main(void) {
94      printf("parent: PID=%d\n", getpid());
95
96      char *name1 = read_filename("File for child1 (lines <= 10 chars): ");
97      char *name2 = read_filename("File for child2 (lines > 10 chars): ");
98
99      int file1 = open_output(name1);
100     int file2 = open_output(name2);
101     free(name1);
102     free(name2);
103
104     int pipe1[2];
```

```
105     int pipe2[2];
106     if (pipe(pipe1) == -1) {
107         die("pipe1");
108     }
109     if (pipe(pipe2) == -1) {
110         die("pipe2");
111     }
112 
113     int close_for_child1[] = { pipe2[0], pipe2[1], pipe1[1], file2 };
114     int close_for_child2[] = { pipe1[0], pipe1[1], pipe2[1], file1 };
115     pid_t child1 = spawn_child(pipe1[0], file1, close_for_child1, 4);
116     printf("parent: started child1, PID=%d\n", child1);
117     pid_t child2 = spawn_child(pipe2[0], file2, close_for_child2, 4);
118     printf("parent: started child2, PID=%d\n", child2);
119 
120     close(pipe1[0]);
121     close(pipe2[0]);
122     close(file1);
123     close(file2);
124 
125     if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
126         die("signal");
127     }
128 
129     int exit_code = EXIT_SUCCESS;
130     char *line = NULL;
131     size_t cap = 0;
132     ssize_t len;
133     while ((len = getline(&line, &cap, stdin)) != -1) {
134         size_t text_len = (size_t)len;
135         if (line[len - 1] == '\n') {
136             text_len--;
137         }
138         int target = text_len > LENGTH_THRESHOLD ? pipe2[1] : pipe1[1];
139         if (write_all(target, line, (size_t)len) == -1) {
140             perror("parent: write to pipe");
141             exit_code = EXIT_FAILURE;
142             break;
143         }
144     }
145     if (ferror(stdin)) {
146         perror("parent: getline");
147         exit_code = EXIT_FAILURE;
148     }
149     free(line);
150 
151     close(pipe1[1]);
152     close(pipe2[1]);
153 
154     if (wait_child(child1, "child1") != 0) {
155         exit_code = EXIT_FAILURE;
156     }
157     if (wait_child(child2, "child2") != 0) {
158         exit_code = EXIT_FAILURE;
159     }
160     return exit_code;
```

```
}
```

Листинг 3: src/parent.c

```c
#include "utils.h"

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    fprintf(stderr, "child: PID=%d\n", getpid());

    char *line = NULL;
    size_t cap = 0;
    ssize_t len;
    while((len = getline(&line, &cap, stdin)) != -1){
        if (line[len-1] == '\n'){
            reverse_string(&line[0], len - 1);
        }
        else{
            reverse_string(&line[0], len);
        }
        if (write_all(STDOUT_FILENO, line, len) == -1) {
        free(line);
        die("write");
        }
    }
    if (ferror(stdin)){
        free(line);
        die("getline");
    }
    free(line);
    return EXIT_SUCCESS;
}
```

Листинг 4: src/child.c

```
cmake_minimum_required(VERSION 3.16)
project(os_lab2 LANGUAGES C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

# compile_commands.json для редактора (clangd / VS Code), чтобы он видел include/
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Debug)
endif()

add_compile_options(-Wall -Wextra -Wpedantic)
# Открывает POSIX-функции (getline и др.) при строгом -std=c11
add_compile_definitions(_POSIX_C_SOURCE=200809L)
```

```

# Общий код для обеих программ
add_library(utils STATIC src/utils.c)
target_include_directories(utils PUBLIC include)

add_executable(child src/child.c)
target_link_libraries(child PRIVATE utils)

add_executable(parent src/parent.c)
target_link_libraries(parent PRIVATE utils)
# Абсолютный путь к собранному child, чтобы parent находил его из любой папки
target_compile_definitions(parent PRIVATE CHILD_PATH="$<TARGET_FILE:child>")
add_dependencies(parent child)
```

Листинг 5: CMakeLists.txt

Strace

Запуск:

```
strace -ff -e trace=execve,openat,read,write,close,pipe2,clone,dup2,wait4,exit_group -
    o trace ./build/parent < input.txt
```

Ключ `-ff` записывает вывод каждого процесса в отдельный файл, ниже они приведены по очереди: родитель, child1, child2.

В родителе видно открытие файлов (`openat`), создание двух каналов (`pipe2`), создание двух процессов (`clone` — так в glibc реализован `fork`), закрытие ненужных дескрипторов, запись коротких строк в дескриптор 6 (`pipe1`), длинных в дескриптор 9 (`pipe2`), закрытие пишущих концов после EOF и ожидание детей (`wait4`). В дочерних процессах видны `dup2`, закрытие лишних дескрипторов, `execve` программы `child`, чтение из канала и запись перевёрнутых строк в файл (дескриптор 1).

```
==== PID 2831
execve("./build/parent", ["./build/parent"], 0x7ffdf6ffa6e8 /* 22 vars */) = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
    = 832
close(3)                                = 0
write(1, "parent: PID=2831\nFile for child1"..., 54) = 54
read(0, "out1.txt\nout2.txt\nhello\nhello wo"..., 4096) = 84
write(1, "File for child2 (lines > 10 char"..., 36) = 36
openat(AT_FDCWD, "out1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
openat(AT_FDCWD, "out2.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
pipe2([5, 6], 0)                        = 0
pipe2([8, 9], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7ba4509e0a10) = 2832
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7ba4509e0a10) = 2833
close(5)                                = 0
close(8)                                = 0
close(3)                                = 0
close(4)                                = 0
write(6, "hello\n", 6)                   = 6
```

```
write(9, "hello world 12345\n", 18)     = 18
write(6, "os lab 2\n", 9)               = 9
write(9, "this line is longer than ten\n", 29) = 29
write(6, "abc\n", 4)                    = 4
read(0, "", 4096)                       = 0
close(6)                                = 0
close(9)                                = 0
wait4(2832, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 2832
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=2832, si_uid=1000, si_status
    =0, si_utime=0, si_stime=0} ---
wait4(2833, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 2833
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=2833, si_uid=1000, si_status
    =0, si_utime=0, si_stime=0} ---
write(1, "parent: started child1, PID=2832"..., 156) = 156
exit_group(0)                           = ?
+++ exited with 0 +++

==== PID 2832
dup2(5, 0)                              = 0
dup2(3, 1)                              = 1
close(5)                                = 0
close(3)                                = 0
close(8)                                = 0
close(9)                                = 0
close(6)                                = 0
close(4)                                = 0
execve("/home/konstsmir/os_labs/lab2/build/child", ["child"], 0x7ffc0d5ba348 /* 22
    vars */) = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
    = 832
close(3)                                = 0
write(2, "child: PID=2832\n", 16)       = 16
read(0, "hello\nos lab 2\nabc\n", 4096) = 19
write(1, "olleh\n", 6)                  = 6
write(1, "2 bal so\n", 9)               = 9
write(1, "cba\n", 4)                    = 4
read(0, "", 4096)                       = 0
exit_group(0)                           = ?
+++ exited with 0 +++

==== PID 2833
dup2(8, 0)                              = 0
dup2(4, 1)                              = 1
close(8)                                = 0
close(4)                                = 0
close(5)                                = 0
close(6)                                = 0
close(9)                                = 0
close(3)                                = 0
execve("/home/konstsmir/os_labs/lab2/build/child", ["child"], 0x7ffc0d5ba348 /* 22
    vars */) = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832)
```

```
    = 832
close(3)                                = 0
write(2, "child: PID=2833\n", 16)       = 16
read(0, "hello world 12345\nthis line is l"..., 4096) = 47
write(1, "54321 dlrow olleh\n", 18)     = 18
write(1, "net naht regnol si enil siht\n", 29) = 29
read(0, "", 4096)                       = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```